

# Seminar 1

Today: start from the **theorem**, leave holes `?_`, and **fill them in** — the way you actually type it in the editor.

And after each proof we'll write down **one short rule**. By the end we'll have our own **rulebook**.

# Plan

1. `implFlipPremises` —  $(A \rightarrow B \rightarrow C) \rightarrow B \rightarrow A \rightarrow C$
2. `implS` —  $(A \rightarrow B \rightarrow C) \rightarrow (A \rightarrow B) \rightarrow A \rightarrow C$
3. `or_elim_tutorial` —  $A \vee B \rightarrow (A \rightarrow C) \wedge (B \rightarrow C) \rightarrow C$
4. `iff_trans` —  $(A \leftrightarrow B) \rightarrow (B \leftrightarrow C) \rightarrow (A \leftrightarrow C)$
5. `iff_and_congr` —  $(A \leftrightarrow B) \rightarrow (C \leftrightarrow D) \rightarrow (A \wedge C \leftrightarrow B \wedge D)$
6. `implPhi` —  $(A \rightarrow B \rightarrow C) \rightarrow (D \rightarrow A) \rightarrow (D \rightarrow B) \rightarrow D \rightarrow C$
7. `contrapositive_double` —  $(A \rightarrow B) \rightarrow \neg\neg A \rightarrow \neg\neg B$
8. `implApplyToChain` —  $((P \rightarrow Q) \rightarrow Q) \rightarrow R \rightarrow (R \rightarrow S) \rightarrow P \rightarrow S$

# The method

For any goal, repeat until there are no holes:

1. **Read** the type of the current hole `?_`.
2. Is it `x → y`? **Introduce** the argument: `fun x => ?_`.
3. **Deconstruct** what you *have* (`λ → ⟨a, b⟩`, `v → cases`).
4. **Build** what you *need* (`λ / ↔ → ⟨?_, ?_⟩`, `v → or.inl / or.inr`).
5. **Match** each `?_` to something in scope.

# implFlipPremises

```
theorem implFlipPremises {A B C : Prop} :  
  (A → B → C) → B → A → C :=  
  ?_
```

1 goal

```
A B C : Prop  
⊢ (A → B → C) → B → A → C
```

# implFlipPremises

```
theorem implFlipPremises {A B C : Prop} :  
  (A → B → C) → B → A → C :=  
  fun f b a => ?_
```

arguments introduced — goal still open

```
f : A → B → C  
b : B  
a : A  
⊢ C
```

# implFlipPremises

```
theorem implFlipPremises {A B C : Prop} :  
  (A → B → C) → B → A → C :=  
  fun f b a => f a b
```

fill the hole

```
f : A → B → C  
b : B ,   a : A  
f a b : C      -- f's order: A then B
```

No goals

Rule 1: `→` groups to the right — `X → Y → Z` means `X → (Y → Z)` ; each top-level left arrow is one `fun` argument.

# implS

```
theorem implS {A B C : Prop} :  
  (A → B → C) → (A → B) → A → C :=  
  fun f g a => ?_
```

1 goal

```
f : A → B → C  
g : A → B  
a : A  
⊢ C
```

# implS

```
theorem implS {A B C : Prop} :  
  (A → B → C) → (A → B) → A → C :=  
  fun f g a => f a (?_)
```

applied `f a` — the hole must be a `B`

```
f : A → B → C  
g : A → B  
a : A  
f a : B → C    -- so the hole needs a B  
⊢ B
```



# implS

```
theorem implS {A B C : Prop} :  
  (A → B → C) → (A → B) → A → C :=  
  fun f g a => f a (g a)
```

filling the hole

```
a    : A          -- for f  
g a  : B          -- uses a again  
f a (g a) : C
```

No goals

# or\_elim\_tutorial

```
theorem or_elim_tutorial {A B C : Prop} :  
  A ∨ B → (A → C) ∧ (B → C) → C :=  
  ?_
```

1 goal — just read the type

```
A B C : Prop  
⊢ A ∨ B → (A → C) ∧ (B → C) → C  
-- have: an A ∨ B and an (A → C) ∧ (B → C)  
-- the second is an ∧ → deconstruct it
```

# or\_elim\_tutorial

```
theorem or_elim_tutorial {A B C : Prop} :  
  A ∨ B → (A → C) ∧ (B → C) → C :=  
  fun h ⟨fa, fb⟩ => ?_
```

write the arguments — `⟨fa, fb⟩` splits the  $\wedge$

```
h   : A ∨ B  
fa  : A → C  
fb  : B → C  
⊢ C
```

# or\_elim\_tutorial

```
theorem or_elim_tutorial {A B C : Prop} :  
  A ∨ B → (A → C) ∧ (B → C) → C :=  
  fun h ⟨fa, fb⟩ =>  
    match h with  
    | Or.inl a => ?_  
    | Or.inr b => ?_
```

2 goals — one per constructor

```
Or.inl a:  a : A , fa : A → C , fb : B → C  
⊢ C
```

```
Or.inr b:  b : B , fa : A → C , fb : B → C  
⊢ C
```

# or\_elim\_tutorial

```
theorem or_elim_tutorial {A B C : Prop} :  
  A ∨ B → (A → C) ∧ (B → C) → C :=  
  fun h ⟨fa, fb⟩ =>  
    match h with  
    | Or.inl a => fa a  
    | Or.inr b => fb b
```

left `fa a` , right `fb b`

```
fa a : C  
fb b : C
```

No goals

Rule 3: if you have an  $\wedge$  or  $\vee$ , deconstruct it ( `⟨a, b⟩` / `match` ).

# iff\_trans

```
theorem iff_trans {A B C : Prop} :  
  (A ↔ B) → (B ↔ C) → (A ↔ C) :=  
  fun hab hbc => ?_
```

arguments introduced — goal still open

```
hab : A ↔ B  
hbc : B ↔ C  
⊢ A ↔ C
```

# iff\_trans

```
theorem iff_trans {A B C : Prop} :  
  (A ↔ B) → (B ↔ C) → (A ↔ C) :=  
  fun hab hbc =>  
    ⟨ ?_ ,  
      ?_ ⟩
```

an  $\leftrightarrow$  is a pair — build `⟨ ?_ , ?_ ⟩`

```
hab.mp   : A → B      hab.mpr : B → A  
hbc.mp   : B → C      hbc.mpr : C → B  
-- goals:  
mp       ⊢ A → C  
mpr      ⊢ C → A
```

# iff\_trans

```
theorem iff_trans {A B C : Prop} :  
  (A ↔ B) → (B ↔ C) → (A ↔ C) :=  
  fun hab hbc =>  
    { fun a => ?_ ,  
      fun c => ?_ }
```

each hole is an arrow → introduce `fun ... => ?_`

```
hab.mp   : A → B      hab.mpr : B → A  
hbc.mp   : B → C      hbc.mpr : C → B  
-- goals:  
mp   a : A ⊢ C  
mpr  c : C ⊢ A
```



# iff\_trans

```
theorem iff_trans {A B C : Prop} :  
  (A ↔ B) → (B ↔ C) → (A ↔ C) :=  
  fun hab hbc =>  
    ⟨ fun a => hbc.mp (hab.mp a),  
      fun c => hab.mpr (hbc.mpr c) ⟩
```

forward direction

```
hab.mp a          : B  
hbc.mp (hab.mp a) : C
```

No goals

Rule 4: an  $\leftrightarrow$  is a pair  $\langle \text{mp}, \text{mpr} \rangle$ ; project with `.mp` / `.mpr`.

# iff\_and\_congr

```
theorem iff_and_congr {A B C D : Prop}
  (hab : A ↔ B) (hcd : C ↔ D) :
  A ∧ C ↔ B ∧ D :=
  ⟨ ?_ ,
    ?_ ⟩
```

an  $\leftrightarrow$  is a pair — build `⟨ ?_ , ?_ ⟩`

```
hab.mp   : A → B      hab.mpr : B → A
hcd.mp   : C → D      hcd.mpr : D → C
-- goals:
mp       ⊢ A ∧ C → B ∧ D
mpr      ⊢ B ∧ D → A ∧ C
```

# iff\_and\_congr

```
theorem iff_and_congr {A B C D : Prop}
  (hab : A ↔ B) (hcd : C ↔ D) :
  A ∧ C ↔ B ∧ D :=
{ fun h1 => ?_ ,
  fun h2 => ?_ }
```

each hole is an arrow → introduce `fun h1 / h2 => ?_`

```
hab.mp   : A → B      hab.mpr : B → A
hcd.mp   : C → D      hcd.mpr : D → C
-- goals:
mp   h1 : A ∧ C ⊢ B ∧ D
mpr  h2 : B ∧ D ⊢ A ∧ C
```

# iff\_and\_congr

```
theorem iff_and_congr {A B C D : Prop}
  (hab : A ↔ B) (hcd : C ↔ D) :
  A ∧ C ↔ B ∧ D :=
  ⟨ fun h1 => ⟨ ?_ , ?_ ⟩ ,
    fun h2 => ⟨ ?_ , ?_ ⟩ ⟩
```

each goal is an  $\wedge$  — build `⟨ ?_ , ?_ ⟩`

```
h1.left : A , h1.right : C
h2.left : B , h2.right : D
-- goals:
mp      ⊢ B , ⊢ D
mpr     ⊢ A , ⊢ C
```

## iff\_and\_congr

```
theorem iff_and_congr {A B C D : Prop}
  (hab : A ↔ B) (hcd : C ↔ D) :
  A ∧ C ↔ B ∧ D :=
  ⟨ fun h1 => ⟨ hab.mp h1.left , ?_ ⟩ ,
    fun h2 => ⟨ ?_ , ?_ ⟩ ⟩
```

first row, left: `hab.mp h1.left : B`

```
h1.left : A , hab.mp h1.left : B
-- remaining:
mp      ⊢ D
mpr     ⊢ A , ⊢ C
```

## iff\_and\_congr

```
theorem iff_and_congr {A B C D : Prop}
  (hab : A ↔ B) (hcd : C ↔ D) :
  A ∧ C ↔ B ∧ D :=
  ⟨ fun h1 => ⟨ hab.mp h1.left , hcd.mp h1.right ⟩ ,
    fun h2 => ⟨ ?_ , ?_ ⟩ ⟩
```

first row done: `⟨ ... ⟩ : B ∧ D`

```
h1.right : C , hcd.mp h1.right : D
-- remaining:
mpr  ⊢ A , ⊢ C
```

## iff\_and\_congr

```
theorem iff_and_congr {A B C D : Prop}
  (hab : A ↔ B) (hcd : C ↔ D) :
  A ∧ C ↔ B ∧ D :=
  ⟨ fun h1 => ⟨ hab.mp h1.left , hcd.mp h1.right ⟩ ,
    fun h2 => ⟨ hab.mpr h2.left , hcd.mpr h2.right ⟩ ⟩
```

second row: mirror with `.mpr`

```
h2.left   : B ,   hab.mpr h2.left   : A
h2.right  : D ,   hcd.mpr h2.right  : C
```

No goals

**Rule 5:** to BUILD an  $\wedge$  /  $\leftrightarrow$ , build each side and combine with `⟨_, _⟩`.

# implPhi

```
theorem implPhi {A B C D : Prop} :  
  (A → B → C) → (D → A) → (D → B) → D → C :=  
  fun f g h d => ?_
```

1 goal

```
f : A → B → C  
g : D → A  
h : D → B  
d : D  
⊢ C
```



# implPhi

```
theorem implPhi {A B C D : Prop} :  
  (A → B → C) → (D → A) → (D → B) → D → C :=  
  fun f g h d => ?_
```

what we can derive

```
g d : A      -- g : D → A  applied to d  
h d : B      -- h : D → B  applied to d  
f needs : A  then  B
```

# implPhi

```
theorem implPhi {A B C D : Prop} :  
  (A → B → C) → (D → A) → (D → B) → D → C :=  
  fun f g h d => f (g d) (h d)
```

fan-out then fan-in

```
g d : A          -- d feeds g  
h d : B          -- d feeds h  
f (g d) (h d) : C
```

No goals

# contrapositive\_double

```
theorem contrapositive_double {A B : Prop} :  
  (A → B) → ¬¬A → ¬¬B :=  
  ?_
```

read the type —  $\neg X$  is  $X \rightarrow \text{False}$

$\neg\neg A = (A \rightarrow \text{False}) \rightarrow \text{False}$

$\neg\neg B = (B \rightarrow \text{False}) \rightarrow \text{False}$

brackets explicit ( $\rightarrow$  groups right):

$(A \rightarrow B) \rightarrow ( \neg\neg A \rightarrow \neg\neg B )$

## contrapositive\_double — the full type

$$(A \rightarrow B) \rightarrow ((A \rightarrow \text{False}) \rightarrow \text{False}) \rightarrow (B \rightarrow \text{False}) \rightarrow \text{False}$$

$\neg X$  is  $X \rightarrow \text{False}$ , so  $\neg\neg A = (A \rightarrow \text{False}) \rightarrow \text{False}$  and  $\neg\neg B = (B \rightarrow \text{False}) \rightarrow \text{False}$ .

## contrapositive\_double

```
theorem contrapositive_double {A B : Prop} :  
  (A → B) → ¬¬A → ¬¬B :=  
  fun f nna nb => ?_
```

extract the three arguments

```
f      : A → B  
nna    : ¬¬A      -- (A → False) → False  
nb     : ¬B       -- B → False  
⊢ False
```

## contrapositive\_double

```
theorem contrapositive_double {A B : Prop} :  
  (A → B) → ¬¬A → ¬¬B :=  
  fun f nna nb => nna ?_
```

apply **nna** — it needs a **¬A**

```
nna : ¬¬A      -- (A → False) → False  
nna wants a  ¬A  (= A → False)  
⊢ A → False
```

## contrapositive\_double

```
theorem contrapositive_double {A B : Prop} :  
  (A → B) → ¬¬A → ¬¬B :=  
  fun f nna nb => nna (fun a => ?_)
```

```
introduce fun a => ?_ (a : A)
```

```
a   : A  
f   : A → B      f a : B  
nb  : ¬B          -- B → False  
⊢ False
```

## contrapositive\_double

```
theorem contrapositive_double {A B : Prop} :  
  (A → B) → ¬¬A → ¬¬B :=  
  fun f nna nb => nna (fun a => nb (f a))
```

inside fun a => ?\_, a:A

```
f a      : B  
nb (f a) : False  
nna (fun a => nb (f a)) : False
```

No goals

Rule 6:  $\neg X$  is  $X \rightarrow \text{False}$ ; a proof of **False** closes any goal.



# implApplyToChain

```
theorem implApplyToChain {P Q R S : Prop} :  
  (((P → Q) → Q) → R) → (R → S) → P → S :=  
  fun h k p => ?_
```

1 goal (optional / hardest)

```
h : ((P → Q) → Q) → R  
k : R → S  
p : P  
-- goals:  
⊢ S
```

# implApplyToChain

```
theorem implApplyToChain {P Q R S : Prop} :  
  (((P → Q) → Q) → R) → (R → S) → P → S :=  
  fun h k p => k ?_
```

apply **k** — it needs an **R**

```
h : ((P → Q) → Q) → R  
k : R → S  
p : P  
-- goals:  
⊢ R
```

# implApplyToChain

```
theorem implApplyToChain {P Q R S : Prop} :  
  (((P → Q) → Q) → R) → (R → S) → P → S :=  
  fun h k p => k (h ?_)
```

apply `h` — it needs a `(P → Q) → Q`

```
h : ((P → Q) → Q) → R  
k : R → S  
p : P  
-- goals:  
⊢ (P → Q) → Q
```

# implApplyToChain

```
theorem implApplyToChain {P Q R S : Prop} :  
  (((P → Q) → Q) → R) → (R → S) → P → S :=  
  fun h k p => k (h (fun f => ?_))
```

the goal is a function → introduce `fun f => ?_`

```
f : P → Q  
p : P  
-- goals:  
⊢ Q
```

# implApplyToChain

```
theorem implApplyToChain {P Q R S : Prop} :  
  (((P → Q) → Q) → R) → (R → S) → P → S :=  
  fun h k p => k (h (fun f => f p))
```

f p closes the last goal

```
f p : Q  
h (fun f => f p) : R  
k (h (fun f => f p)) : S
```

No goals

Rule 7: stuck on  $X \rightarrow Y$  (even  $(P \rightarrow Q) \rightarrow Q$ )? Introduce `fun x => ?_`.

# Our rulebook

1. `→` groups to the right ( `x → y → z = x → (y → z)` ); each top-level left arrow is one `fun` argument.
2. A term may be used **more than once**.
3. If you have an `∧` or `∨`, **deconstruct** it ( `<a, b>` / `match` ).
4. An `↔` is a pair `<mp, mpr>`; project with `.mp` / `.mpr`.
5. To **build** an `∧` / `↔`, build each side and combine with `<_, _>`.
6. `¬x` is `x → False`; a proof of `False` closes any goal.
7. Stuck on `x → y`? Introduce `fun x => ?_`.